

Dino Game Bot

Deep Dive

Explainer document generated from the project source

What this document is

An exhaustive, beginner-safe walkthrough of `dino-game-bot`, a Python script that automates Chrome's offline Dino game by watching the screen and pressing space. Every function, constant, and design decision in `main.py` and the `tools/` scripts is covered, with every technical term defined at first use. See `brief.pdf` in the same folder for a shorter, higher-level version. License note: this project is under PolyForm Strict 1.0.0, which lets anyone view the source but grants no rights to copy, modify, or redistribute it. It is not MIT or any other permissive license.

Contents

1	What the project does, and why	2
2	Architecture overview	2
3	Detection logic	3
3.1	Constants	3
3.2	<code>check_day(tolerance=0)</code>	3
3.3	<code>check_obstacle(is_day=None, tolerance=0)</code>	3
3.4	Decision loop	3
4	Status dashboard	3
5	Demo mode	3
5.1	Why it exists	4
5.2	How frames reach the real detection code unmodified	4
5.3	Synthetic frames: <code>tools/generate_demo_frames.py</code>	4
5.4	<code>DemoTkStatusView</code> and <code>DemoConsoleStatusView</code>	4
5.5	CLI	5
6	Detection robustness experiment	5
6.1	The problem being tested	5
6.2	What was added	5
6.3	Proving it before adopting it	5
6.4	What this does and does not mean	5
7	Verification actually performed	6

8 License

6

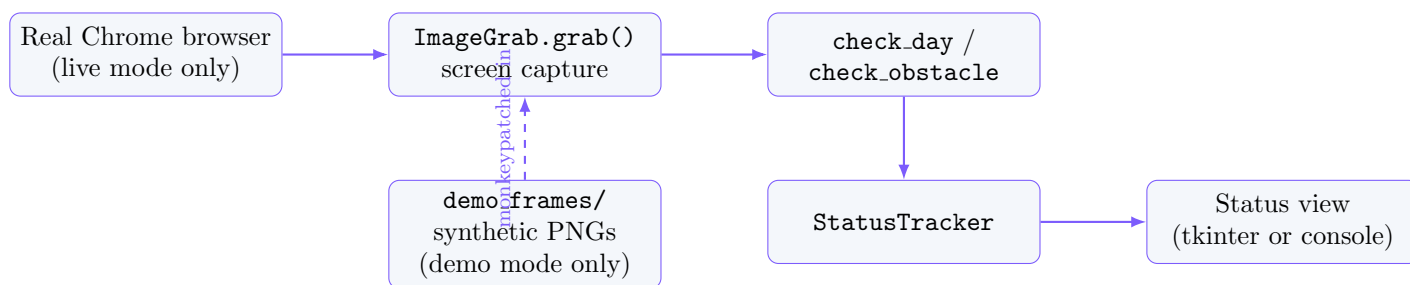
1 What the project does, and why

The Dino game is Chrome’s built-in offline minigame (`chrome://dino/`): a pixel-art dinosaur runs across the screen automatically, and the only input is “jump” (space bar), timed to clear cactus and bird obstacles. This bot plays it by treating the problem as pure computer vision plus keyboard automation: it never reads the game’s internal state or its HTML/DOM, only pixels it captures from the screen, and it only ever sends a single keypress (space) back.

Glossary, defined once here and used throughout:

- **Selenium / WebDriver**: a library and browser-automation protocol that lets a script drive a real browser window programmatically (open pages, click, send keys) as if a person were using it.
- **Screen capture / `ImageGrab.grab()`**: taking a screenshot of a rectangular region of the physical screen and getting it back as an in-memory image, independent of what application drew those pixels.
- **RGB tuple**: a pixel’s color as three numbers 0–255, one each for red, green, blue, e.g. (255, 255, 255) is pure white.
- **Bounding box (bbox)**: a rectangle described as (left, top, right, bottom) in pixel coordinates.
- **tkinter**: Python’s built-in GUI toolkit, used here to draw the status dashboard window.
- **Monkeypatching**: replacing a function or attribute at runtime with a different implementation, without editing the code that calls it. Used here to swap the screen-capture function for a stand-in during demo mode.
- **CLI (command-line interface) argument**: a flag typed after the program name, e.g. `--demo`, parsed here with Python’s built-in `argparse` module.
- **Color tolerance / color distance**: instead of requiring a pixel to be an exact color match, accepting it as a match if every red/green/blue channel is within some small numeric distance of the target color.

2 Architecture overview



Two entry points share almost everything: live mode (`main()`) drives a real Selenium-controlled Chrome window and reads real screen pixels; demo mode (`run_demo()`) reads pre-drawn PNG frames instead, but pushes them through the exact same `check_day/check_obstacle` functions by substituting what `ImageGrab.grab()` returns, not by branching inside the detection logic itself.

3 Detection logic

3.1 Constants

```
OBSTACLE_AREA_COORDINATES = (380, 645, 950, 760)
GAME_AREA_COORDINATES = (20, 580, 800, 760)
DAY_GROUND_COLOR = (255, 255, 255)
NIGHT_GROUND_COLOR = (83, 83, 83)
```

These are absolute screen pixel coordinates, tuned for one specific maximized Chrome window on one specific screen resolution; they need to be re-measured on any other display. `GAME_AREA_COORDINATES` is a wide box covering ground and sky, used to decide day vs. night. `OBSTACLE_AREA_COORDINATES` is a narrower box positioned just ahead of the dino, used to decide whether to jump right now.

3.2 `check_day(tolerance=0)`

Grabs the game-area box and counts how many pixels equal `DAY_GROUND_COLOR` versus `NIGHT_GROUND_COLOR`; whichever count wins is the current mode. At the default `tolerance=0` this is an exact tuple count, byte for byte identical to the original implementation. Passing `tolerance > 0` instead counts any pixel whose color is within that per-channel distance of each target color (see §6).

3.3 `check_obstacle(is_day=None, tolerance=0)`

Grabs the narrower obstacle-area box. In day mode, obstacles render as the dark "night" color against the light background, so it checks whether `NIGHT_GROUND_COLOR` appears anywhere in that box; in night mode the colors invert and it checks for `DAY_GROUND_COLOR` instead. This is why `check_day` has to run first: the obstacle color to look for depends on the current mode. The optional `is_day` argument lets a caller that already computed the mode (e.g. the main loop, for the dashboard) skip a second screen grab. At `tolerance=0` the obstacle check is exact membership (`target in pixels`), identical to the original code.

3.4 Decision loop

On every iteration, `main()` calls `check_day()`, then `check_obstacle(is_day)`. If an obstacle is present, it immediately sends a space keypress. There is no distance or speed estimate: the obstacle box is placed close enough to the dino that "obstacle present in the box" and "jump now" are the same event.

4 Status dashboard

`StatusTracker` is a plain, display-independent object updated once per loop iteration (`record(is_day, obstacle_detected, jumped)`); it tracks mode, obstacle flag, jump count, elapsed time, and a rolling log of the last 8 events. `build_status_view()` picks a renderer: a non-blocking `tkinter` window (`TkStatusView`) if a display is available, or a throttled console printer (`ConsoleStatusView`) if constructing the window raises anything (no `DISPLAY`, headless CI, Tk missing). Neither of these two classes was modified for this round of work; demo mode uses subclasses instead (§5), so live mode's dashboard behavior is unchanged.

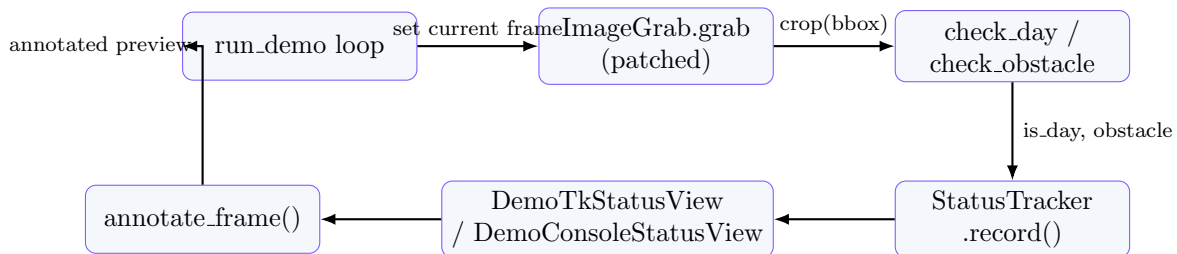
5 Demo mode

5.1 Why it exists

There is no live Chrome window actually playing the Dino game in every environment this project runs in (a fresh clone, this project’s own review environment, CI). Demo mode makes the detection logic independently checkable without one: it replays a small set of hand-drawn screenshots through the real `check_day/check_obstacle` functions and shows, frame by frame, exactly what the bot concluded and why.

5.2 How frames reach the real detection code unmodified

`check_day` and `check_obstacle` call `ImageGrab.grab(bbox=...)` exactly as before; neither function was changed to add a “demo” branch. Instead, `run_demo()` temporarily replaces the module-level `ImageGrab.grab` function with a small stand-in that crops the current demo frame instead of capturing the screen, runs one frame through the unmodified detection functions, then restores the original `ImageGrab.grab` in a `finally` block once all frames are done (or the dashboard window is closed early).



5.3 Synthetic frames: tools/generate_demo_frames.py

This script imports `DAY_GROUND_COLOR`, `NIGHT_GROUND_COLOR`, `GAME_AREA_COORDINATES`, and `OBSTACLE_AREA_COORDINATES` directly from `main.py` (rather than hardcoding copies) so the frames can never silently drift out of sync with the real detection constants. It draws six 1050×800 pixel PNGs, each a solid fill of the day or night ground color with an optional obstacle rectangle (rendered in the opposite mode’s color, which is exactly the color `check_obstacle` scans for):

Filename	Mode	Obstacle
01_day_clear.png	day	none
02_day_obstacle_far.png	day	right of the trigger box
03_day_obstacle_trigger.png	day	inside the trigger box
04_night_clear.png	night	none
05_night_obstacle_far.png	night	right of the trigger box
06_night_obstacle_trigger.png	night	inside the trigger box

The “far” frames matter because they exercise a real distinction: an obstacle can be visible on screen without yet being in the narrow trigger zone, and the bot must not jump early.

5.4 DemoTkStatusView and DemoConsoleStatusView

These are new subclasses of the existing `TkStatusView/ConsoleStatusView`, not modifications of them, so live mode’s dashboard is untouched. `DemoTkStatusView` adds one extra widget: an image panel showing the current frame with `annotate_frame()`’s bounding boxes burned in (blue for the day/night sample box, green or red for the obstacle box depending on whether that frame’s `check_obstacle` call returned true). `build_status_view(demo=True)` selects these variants; the default `demo=False` call used by `main()` is exactly the code path that existed before this work.

5.5 CLI

```
python3 main.py                # live mode, unchanged
python3 main.py --demo         # replay demo_frames/ through real detection
python3 main.py --demo --demo-dir path/ --demo-delay 0.5
```

6 Detection robustness experiment

6.1 The problem being tested

The original detection code requires an exact RGB tuple match. That is airtight against a hand-drawn synthetic frame (which is a perfectly flat, exact color by construction) but is a real fragility against anything with color drift a real screen capture can have: anti-aliased sprite edges, dithering, or lossy compression, none of which a synthetic solid-color frame can ever demonstrate on its own.

6.2 What was added

`check_day` and `check_obstacle` both gained an optional `tolerance` parameter, default 0. At 0 the code path is byte-for-byte the original exact-match logic (verified by re-running the demo frame set unchanged, §7). At `tolerance > 0`, a pixel counts as a match if every channel is within that distance of the target color, via a small helper:

```
def _color_distance(pixel, target):
    return max(abs(pixel[i] - target[i]) for i in range(3))
```

6.3 Proving it before adopting it

`tools/compare_detection_robustness.py` builds a second, "noisy" copy of every demo frame: every pixel is nudged ± 6 per channel in a deterministic checkerboard pattern (clipped to 0–255), so no pixel in the noisy set is ever exactly equal to the clean frame's ground/obstacle color, standing in for the kind of small unavoidable capture noise a synthetic frame cannot otherwise produce. It then runs both the old exact-match logic (`tolerance=0`) and the new tolerant logic (`tolerance=8`) against both the clean and noisy frame sets and checks each result against the ground truth encoded in each filename.

Real output from this script (§7 has the full transcript):

	clean frames	noisy frames
old (tolerance=0)	6/6 correct	4/6 correct
new (tolerance=8)	6/6 correct	6/6 correct

The old exact-match logic loses two of six results on the noisy set: it misses the obstacle entirely on one trigger frame, and misreads night as day on another (because dithering pushes every ground pixel just off the exact target color). The tolerant logic recovers full accuracy on the noisy set with zero regression on the clean set, which is the bar this project's standards require before adopting a change like this: proven with a real before/after run, not assumed.

6.4 What this does and does not mean

This is not evidence about real Chrome Dino gameplay, only about robustness to a specific, deliberately constructed kind of color noise. `tolerance` defaults to 0 everywhere it is called from `main()` and `run_demo()`, so this change does not alter any currently observed behavior; it is available for a future

caller (or a future default change) once it has been exercised against a real capture, which has not happened here.

7 Verification actually performed

Verified in this environment (real commands, real output)

- `python3 -c "import ast; ast.parse(open('main.py').read())"` parses with no errors.
- `python3 tools/generate_demo_frames.py` wrote all 6 frames listed above to `demo_frames/`.
- `python3 main.py --demo` was run against a real X display in this environment: the tkinter dashboard constructed without exceptions, and every frame produced the expected decision (day/night correctly identified, jump triggered only on the two "trigger" frames, not on the "far" frames).
- The same run was repeated with `DISPLAY` unset, confirming the console fallback (`DemoConsoleStatusView`) activates and prints the same correct per-frame decisions headlessly.
- `tools/compare_detection_robustness.py` was run and produced the 6/6 vs. 4/6 vs. 6/6 vs. 6/6 table above from a real execution, not estimated numbers.
- After a demo run, `ImageGrab.grab` was confirmed restored to the real function (not left monkeypatched), and `build_status_view()` with no arguments was confirmed to still construct a plain `TkStatusView`, not a demo variant.

Not verified here, still needs a real machine with Chrome

- Actually launching Chrome, navigating to `chrome://dino/`, and confirming the game starts and responds to a real jump keypress.
- Whether `OBSTACLE_AREA_COORDINATES` and `GAME_AREA_COORDINATES` still match a real dino game window on any given screen; they were not re-measured here and are unrelated to the demo mode work.
- End-to-end play against the real game: whether the bot clears obstacles reliably over a real run, with or without the tolerance-based matching.
- Whether `tolerance > 0` helps or hurts against real screen-capture noise specifically (as opposed to the synthetic dithering pattern used here), since that has only been tested against a deliberately constructed approximation of capture noise, not a real capture.

8 License

This project is distributed under PolyForm Strict 1.0.0. That license permits viewing the source but grants no rights to copy, modify, or redistribute it. It is not MIT, BSD, Apache, or any other permissive or copyleft open-source license.